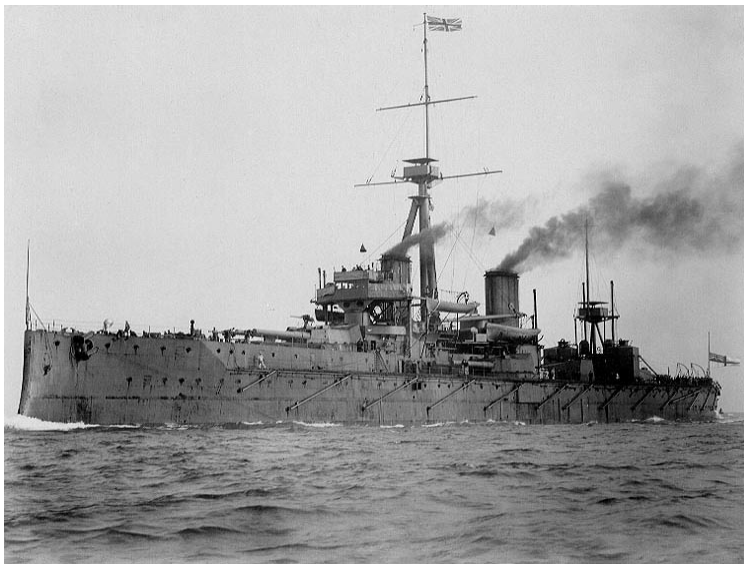
Lab Assignment #4: Representation Learning via Modern RL



1 Introduction

One of the major developments in reinforcement learning has been the replacement of manually engineered state representations with learned representations. Classical RL relies on the user to define useful state information for the agent, whereas deep RL attempts to learn representations of state information directly from observations. Neither approach is universally superior.

The goal of this assignment is to investigate experimentally when representation learning via DQN provides a meaningful advantage over the combination of carefully engineered features and classical Q-learning. We will use these algorithms to build agents capable of playing the game of Battleship. Battleship is partially observable; actions in the current turn reveal information about the hidden state which becomes available for the agent to use in future turns.

2 Set up and learn about the environment

In the original board game, two players each take control of a fleet of five ships of different sizes, arranged in secret on a 10×10 grid. They then take turns selecting a grid cell on their opponent's grid on which to fire. If the cell has a part of one of their opponent's ships in it, they learn that they have a hit. Otherwise they learn that they have a miss.

Once all locations on a ship are hit it is sunk and removed from the board. Play continues in this way until one player has had all of their ships sunk.

Our version of Battleship will simplify the classic version in several ways:

- We will work with a one-player version of the game in which there is a hidden board and only the agent gets to select targets;
- The hidden fleet will have three ships, one two grid cells long, one three grid cells long, and one four grid cells long;
- The grid will be 6×6 cells;

Although coding up a simulator is an important part of reinforcement learning we have already had opportunities to practice this; I will provide code that implements the Battleship game so that the focus can be on training agents to play.

1. Download the Battleship_Simulator.ipynb file. This contains a class that sets up and runs the simplified Battleship game.
2. Set up the environment class by specifying the appropriate config dictionary:

```
config = {'n':6,'ships':[2,3,4]}
```

3. Simulate 1000 games with randomly generated actions.

3 Training Agents to play Battleship...

In this part of the lab we will construct multiple agents capable of managing our Battleship environment and compare/contrast their performance.

3.1 ...with Classical Reinforcement Learning

1. **Decide how you will represent the state.** Remember – this will be the only information that the agent has available to make decisions. Battleship is a partially observable Markov Decision Process, since the disposition of the enemy fleet is not known by the agent. The simplest way to represent the state would be to record, for each grid cell, whether it is a hit, a miss, or has not yet been targeted. This version of the state is how self.state in the environment is currently set up. Representing the state in this way is unlikely to work – you will need to develop an alternative conceptualization for the state for your agent to use. Think creatively! What information does your agent need to manage their choices effectively? And then, given how you represent the state, you will need to align the actions of the agent to the appropriate level of granularity.

2. **Choose a reward function.** Battleship is, at its heart, a race. The goal of the agent is to win the game quickly and you need a reward function, $r(s', a, s)$, to incentivize the agent to do this. There are many possible reward functions. Perhaps the simplest would be to record the total number of hits or the total number of hits less the total number of misses (but be careful!).

3. **Implement classical RL algorithms.** Build an agent class that includes at a minimum the Q-values, the policy, and the learning update rule for the Q-learning algorithm from lecture.

3.2 ...with DQN

1. **Use the current board as the state.**

2. **Choose a reward function.** You may use the same reward function as in section 3 if you like, though there may be more information to work with this time.

3. **Implement DQN.** Build an agent class that includes at a minimum the Q-values, the policy, and the learning update rule for the DQN algorithm from lecture.

3.3 Train your agents

1. Train two agents using classical Q-learning. Use the current board as the state for the first of these agents. Use your hand-built state representation for the second agent. You will need to choose the probability of exploring ε, the discount factor γ, and the learning rate, α.

2. Train an agent using DQN. You will need to choose the probability of exploring ε, the discount factor γ, the learning rate, α, the batch size, B, the size of the replay buffer, M, and the frequency with which the target approximator is updated, C.

3. Save the three agents after approximately 500, 5,000, 10,000, and 20,000 training episodes (or another reasonable schedule). Record average reward and average turns to completion during training.

4 Ablations!

In the next part of the lab we will use the code you have developed to assess what elements of the state/algorithm are most important.

1. For your engineered representation, remove one feature at a time and measure the effect on sample efficiency and final performance.

2. Perform a similar ablation study for DQN by removing or modifying one major component (such as the replay buffer, target network, or layers in your deep learning model). Measure the effect on sample efficiency and final performance.

Lab Assignment Report

Write up your analysis in a separate report of no more than 2 pages. Your report should answer the following questions:

1. Describe in detail how the Battleship works and what each method within it does.
2. Provide and interpret the plot of the distribution over game length that you created in 2.3.
3. Describe how you represented the state in 3.1.1 for classical Q-learning. How many total states are there given your representation of the state?
4. Describe the reward function you chose in 3.1.2 and 3.2.2. How do you think it impacted the learning of your agents?
5. What do the learned policies of the agents look like? Specifically, create and provide a visualization of what the agent chooses to do given the board state using target-selection heatmaps, representative games, action distributions, estimated value functions, or another meaningful visualization. Compare the learned strategies and explain how they differ. Have you trained qualitatively different agents via these different algorithms?
6. What effect (if any) do γ and ε have on the performance of these agents? How would you assess this?
7. Compare the performance of your agents. Do they do substantially better than randomly chosen actions? Discuss which representation learns fastest and whether this relationship changes later in training.
8. Identify which features contribute most to successful learning. Explain why each component influences learning.
9. Use an AI assistant to analyze one of your experiments. Ask it why one agent outperformed another. Critically evaluate the explanation by identifying what the AI explained correctly, what it omitted, any unsupported conclusions, and evidence from your experiments that supports or contradicts its analysis.

5 Submission Instructions

- Write up the answers to the questions in a short word document; aim for no more than 2 pages of written text and include all graphics generated. Add footnotes identifying which sentence addresses which questions. Write in complete sentences organized into paragraphs – your goal is to explain what you've done and what you've learned to your audience (me!). Include the appropriate plots you've generated as mentioned above. Convert this to pdf and submit it. Submit your .ipynb file and an .html of it as well.

- Please use the following naming conventions:
  - Lab_4_[YOUR NAME].pdf
  - Lab_4_[YOUR NAME].html
  - Lab_4_[YOUR NAME].ipynb
- The grading rubric for this assignment will be available in Canvas.
- NO OTHER SUBMISSION TYPES WILL BE ACCEPTED.
- You are welcome to use AI tools for implementation as you code up your solution, but the emphasis is on experimental design, analysis, and scientific reasoning. If you do use AI you must include all prompts used as an appendix in your written report or provide a shareable link to them. Any uncited use of generative AI (i.e. prompts not provided) will be considered plagiarism. You may not use generative AI to write your report.